Lecture 2

x86 Processor Architecture — Lecture Notes

I. **General Concepts.** Microcomputer design, buses, clock, the instruction execution cycle, pipelining, reading from memory, cache, multitasking.

II. **IA-32 Processor Architecture.** Intel history, modes, CISC vs. RISC.

III. **Registers.** General-purpose, segment, index and special registers, and the status flags.

IV. **Memory Management.** Real-address mode, segment:offset addresses, protected mode.

Notation. [S39] means slide 39 of Lecture 2.

Mahmoud AboZied
Teaching Assistant

Academic Year 2026 / 2027

Contents

Part I · General Concepts **3**
1 · Basic Microcomputer Design [S3, 4] 3
2 · The Clock [S5] 3
3 · Instruction Execution Cycle [S6, 7] 3
4 · Pipelining & Superscalar [S8, 9, 10, 11] 3
5 · Reading from Memory [S12] 4
6 · Cache Memory [S13] 4
7 · Multitasking & the OS [S14, 15] 4

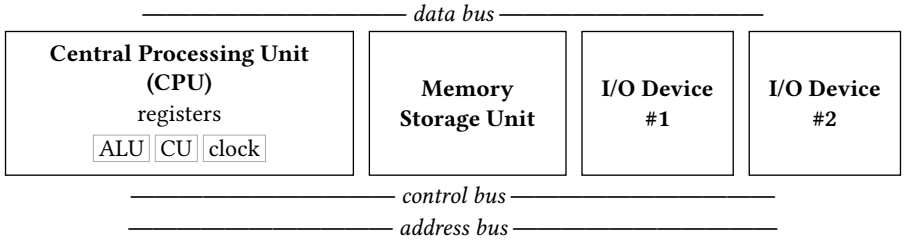
Part II · IA-32 Processor Architecture **5**
8 · Intel Microprocessor History [S16, 17, 18] 5
9 · CISC vs. RISC [S19] 5

Part III · Registers **6**
10 · What Are Registers? [S20] 6
11 · General-Purpose Registers [S21, 23, 24, 25, 32, 33, 34] 6
12 · Accessing Parts of a Register [S22, 23] 6
13 · Segment Registers [S26] 6
14 · Special Registers & Flags [S27, 28, 29, 30] 7
15 · Summary of Registers [S34] 7

Part IV · Memory Management **8**
16 · Addressable Memory [S35, 36] 8
17 · Real-Address Mode [S37, 38] 8
18 · Calculating Linear Addresses [S39] 8
19 · Protected Mode [S40, 41] 8
20 · Real Mode vs. Protected Mode [S42] 8
Key Points to Remember 9

Part I · General Concepts

1 · Basic Microcomputer Design [S3, 4]



Inside the CPU

- **Clock**: synchronizes CPU operations.
- **Control unit (CU)**: coordinates the sequence of execution steps.
- **ALU**: performs arithmetic and bitwise processing.
- **Registers**: fast storage inside the CPU.

Outside the CPU

- **Memory storage unit**: primary memory.
- **I/O devices**: input/output devices.
- **Data bus**: moves data between memory/I/O and the registers.
- **Control bus**: determines where data comes from and goes, and the ALU activities.
- **Address bus**: selects **where** data comes from or goes to.

2 · The Clock [S5]

- Synchronizes **all** CPU and bus operations.
- A **machine (clock) cycle** measures the time of a **single operation**.
- The clock is used to **trigger events**.

3 · Instruction Execution Cycle [S6, 7]

Basic steps

1. Fetch instruction
2. Decode
3. Fetch operands (memory ⟶ registers)
4. Execute
5. Store output

The loop

1. **Fetch** the next instruction.
2. **Increment** the program counter (PC / IP).
3. **Decode** the instruction.
4. If a memory operand is needed, **fetch** it from memory.
5. **Execute** the instruction.
6. If the result is a memory operand, **store** it in memory.

Continue the loop.

The **core** is **fetch ⟶ decode ⟶ execute**.

4 · Pipelining & Superscalar [S8, 9, 10, 11]

Processor	Idea	Cycles for k stages, n instructions
Non-pipelined	Each instruction passes through all stages before the next one starts. **Many wasted cycles.**	**k × n**

Processor	Idea	Cycles for k stages, n instructions
Pipelined	Execution is split into discrete stages, so instructions run **in parallel**. If one stage needs **two** cycles, cycles are wasted again.	**k + (2n – 1)**
Superscalar	**Multiple execution pipelines** (e.g. stage S4 has two pipelines, **u** and **v**).	**k + n**

Example: 6 stages, 2 instructions ⟶ non-pipelined 12 cycles, pipelined (one 2-cycle stage) 6 + 3 = 9 cycles, superscalar 6 + 2 = 8 cycles.

5 · Reading from Memory [S12]

Memory responds **much more slowly** than the CPU, so a read takes **several machine cycles**:

Cycle	What happens
1	The **address** is placed on the address bus.
2	The read line (**RD**) is set **low** (0).
3	The CPU **waits** one cycle for memory to respond.
4	RD goes back to **1**: the **data** is on the data bus.

Registers are **inside** the CPU, so register access is much faster than memory access.

6 · Cache Memory [S13]

- **High-speed, expensive static RAM**, both inside and outside the CPU.
  - **Level-1** cache: **inside** the CPU.
  - **Level-2** cache: **outside** the CPU.
- **Cache hit**: the data to be read is **already** in the cache.
- **Cache miss**: the data to be read is **not** in the cache.

7 · Multitasking & the OS [S14, 15]

Multitasking

- The OS runs **multiple programs** at the same time, and multiple **threads** within one program.
- The **scheduler** gives each running program a slice of CPU time.
- Rapid switching gives the **illusion** that all programs run at once. The processor must support **task switching**.

The OS when you run a program

- Locates the binary executable file on disk.
- Calls the **loader** to move it from disk to memory and resolve addresses.
- Locates and links runtime **DLLs**.
- During execution: resolves addresses, allocates run time (scheduler), manages I/O requests.
- **Cleans up** after execution.

Part II · IA-32 Processor Architecture

8 · Intel Microprocessor History [S16, 17, 18]

Processor	Year	Key feature
8086 / 8088	1978 / 1979	First IBM-PC. **16-bit registers**, 16-bit data bus, 20-bit address bus (1 MB).
80386	1985	**32-bit registers** (IA-32).
80486	1989	Instruction **pipelining**.
Pentium (P5)	1993	**Superscalar** (multiple ALUs).

- **Intel64 mode**: 64-bit linear address space (Pentium Extreme, Xeon, Celeron D, Pentium D, Core 2, Core i3/i5/i7).
- **IA-32e mode** has 2 sub-modes: **compatibility mode** for legacy 16/32-bit applications, and **64-bit mode** with 64-bit addresses and operands.

9 · CISC vs. RISC [S19]

	CISC — complex instruction set	RISC — reduced instruction set
Instruction set	**Large**	**Small**
Instructions	**High-level** operations	Simple, **atomic** instructions
Execution	Requires a **microcode interpreter**	**Directly executed by hardware**
Example	Intel 80x86 family	ARM (Advanced RISC Machines)

Part III · Registers

10 · What Are Registers? [S20]

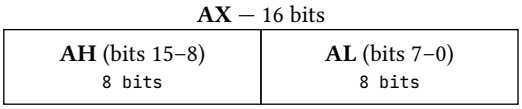
Registers are **high-speed storage locations directly inside the CPU**, designed to be accessed much faster than conventional memory. Types: general-purpose registers · segment registers · the processor status flags register · the instruction pointer.

11 · General-Purpose Registers [S21, 23, 24, 25, 32, 33, 34]

32-bit	16-bit	8-bit	Name	Use
EAX	**AX**	AH, AL	Accumulator	Accumulates results of arithmetic & logic
EBX	**BX**	BH, BL	Base	Points to memory locations (arrays)
ECX	**CX**	CH, CL	Count	Loop counter (auto increment/decrement)
EDX	**DX**	DH, DL	Data	Data used in calculations
ESI	SI	—	Source index	Start address of a source string/array
EDI	DI	—	Destination index	Start address of a destination string/array
EBP	BP	—	Base pointer	Stack-based pointer
ESP	SP	—	Stack pointer	Top of the stack

Primarily used for **arithmetic and data movement**. ESI, EDI, EBP and ESP have only a **16-bit** name for their lower half: no 8-bit halves. [S33]

12 · Accessing Parts of a Register [S22, 23]



Use the **8-bit**, **16-bit** or **32-bit** name. This applies to EAX, EBX, ECX and EDX.

- **EAX**: the full 32-bit value.
- **AX**: the **lower 16** bits of EAX.
- **AL**: the **lower 8** bits (bits 7–0).
- **AH**: the **high** bits 8 through 15.

Example. If `AX = 1234h` then `AH = 12h` and `AL = 34h`. AX is AH and AL **side by side** (AX = AH × 100h + AL), not AH + AL.

13 · Segment Registers [S26]

Register	Name	Points to
CS	Code segment	The memory area holding the program's **instructions**
DS	Data segment	The memory area holding the program's **data**
SS	Stack segment	The memory area holding the **stack**
ES, FS, GS	Extra segments	Additional data segments, if necessary

All segment registers are **16-bit**. The 8086 has **four** (CS, DS, SS, ES). FS and GS came with the 80386.

14 · Special Registers & Flags [S27, 28, 29, 30]

- **EIP** (instruction pointer): the address of the **next instruction** to be executed.
- **EFLAGS**: status and control flags. Each flag is a **single binary bit**. The 8086 has a 16-bit FLAGS register.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
–	–	–	–	**O**	**D**	**I**	**T**	**S**	**Z**	–	**A**	–	**P**	–	**C**

Flag	Name	Set to 1 when ...	Example (8-bit)
CF	Carry	carry or borrow: **unsigned** result out of range	`FFh + 01h` ⟶ CF = 1
OF	Overflow	**signed** result out of range	`7Fh + 01h` ⟶ OF = 1
SF	Sign	the result is **negative** (MSB = 1)	`F0h + 0Fh` ⟶ SF = 1
ZF	Zero	the result is **zero**	`55h - 55h` ⟶ ZF = 1
AF	Auxiliary carry	carry from **bit 3 to bit 4**	`0Fh + 01h` ⟶ AF = 1
PF	Parity	the least-significant byte of the result has an **even** number of 1 bits	`03h` ⟶ PF = 1
IF	Interrupt	an interrupt is currently active (enabled)	—
—	Supervisor	the CPU is operating in supervisor mode	—

Example: the zero flag [S31]

```
            ; initially, assume ZF = 0
mov al, 55h ; ZF is still 0    (MOV does not change flags)
sub al, 55h ; result is 0      → ZF = 1
mov cx, 0   ; ZF remains 1
inc cx      ; result is 1      → ZF = 0
```

15 · Summary of Registers [S34]

32-bit	Name	16/8-bit parts	Primary use
eax	Accumulator	ax, ah, al	Arithmetic and logic
ebx	Base	bx, bh, bl	Arrays
ecx	Counter	cx, ch, cl	Loops
edx	Data	dx, dh, dl	Arithmetic
esi	Source index	si	Strings and arrays
edi	Destination index	di	Strings and arrays
esp	Stack pointer	sp	Top of stack
ebp	Base pointer	bp	Stack base
eip	Instruction pointer	ip	Points to the next instruction
eflags	Flags	flags	Status and control flags

Part IV · Memory Management

16 · Addressable Memory [S35, 36]

16-bit	**20-bit**	**2^{20} B = 1 MB**	**4 GB**
8086 registers & data bus	*address (real mode)*	*real-mode memory*	*protected mode, 32-bit address*

17 · Real-Address Mode [S37, 38]

- **1 MB** RAM maximum addressable (00000h – FFFFFh).
- Application programs can access **any** area of memory.
- **Single tasking**. Supported by **MS-DOS**.

Segmented memory: the 8086's 16-bit registers could not hold 20-bit addresses. So the **linear (absolute) address** is a combination of a **16-bit segment** value and a **16-bit offset**. A segment value is stored with its **last hex zero dropped**.

18 · Calculating Linear Addresses [S39]

Formula.

Linear address = Segment × 10h + Offset

Multiply the segment by 10h (**add a hexadecimal zero** at the end), then add the offset.

```
Convert 08F1:0100
Adjusted segment:  0 8 F 1 0
Add the offset:      0 1 0 0
Linear address:    0 9 0 1 0
```

```
Convert 1000:0020
Adjusted segment:  1 0 0 0 0
Add the offset:      0 0 2 0
Linear address:    1 0 0 2 0
```

More examples: `2000:0100` ⟶ **20100h** · `FFFF:0000` ⟶ **FFFF0h** · the highest address is `FFFF:000F` ⟶ **FFFFFh**.

19 · Protected Mode [S40, 41]

- **4 GB** addressable RAM (00000000h – FFFFFFFFh), 32-bit addresses.
- Each program is assigned a **memory partition** that is **protected** from other programs.
- Designed for **multitasking**. Supported by **Linux & MS-Windows**.
- Memory models: **flat segment** model, **multi-segment** model, **paging**.

20 · Real Mode vs. Protected Mode [S42]

	Real-address mode	**Protected mode**
Registers / addresses	16-bit registers, 20-bit address	32-bit
Addressable memory	**1 MB**	**4 GB**
Tasking	Single task	Multitasking

	Real-address mode	**Protected mode**
Memory access	Any program can access any area	Each program has a protected partition
OS	MS-DOS	Windows, Linux

Note: in the comparison table on slide 42 the "addressable memory" row is swapped. The correct values (from slides 36, 37 and 40) are shown above.

Key Points to Remember

Topic	**Remember**
CPU parts	Registers + ALU + CU + clock
Buses	Data (what) · Address (where) · Control (how)
Execution cycle	Fetch ⟶ (increment IP) ⟶ decode ⟶ fetch operands ⟶ execute ⟶ store
Cycles	Non-pipelined $k \cdot n$ · pipelined $k + (2n - 1)$ · superscalar $k + n$
Memory read	Several cycles. Registers are faster than memory
Cache	L1 inside, L2 outside · hit = in cache, miss = not in cache
CISC / RISC	x86, microcode / ARM, direct hardware
AX	AH (bits 15–8) : AL (bits 7–0), 16 bits. EAX = 32 bits
AX BX CX DX	Accumulator · Base · Count · Data
CS DS SS ES	Code · Data · Stack · Extra
IP / EIP	Address of the next instruction
Flags	CF unsigned · OF signed · SF negative · ZF zero · AF bit 3⟶4 · PF even parity
Linear address	Segment × 10h + Offset
Modes	Real: 1 MB, single task, DOS · Protected: 4 GB, multi-task, Windows/Linux

— End of Lecture 2 —